Sprint 15 · Reto «El mejor modelo, demostrado» · Entrega y rúbrica

TABLA A · LOS FICHEROS

qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el cuaderno) y **cuáles se suben y adónde**.

TABLA B · LOS REQUISITOS

qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

EN TOTAL, AL FINAL DEL SPRINT SE SUBEN A `SPRINT-15/`

6 ficheros del equipo (`registro.csv`, `03_modelos.ipynb`, `justificacion_candidato.md`, `modelo_candidato.joblib`, `metadatos.json`, `explicacion_modelo.png`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **6 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 9). «El reto» no es un fichero aparte: son esos seis ficheros. Los datos ya están en el repositorio desde el Sprint 14 y no se vuelven a subir. La *review* y la *retro* se hacen de palabra: no se suben. Si vuestro modelo guardado pesa más de 25 MB, el navegador no lo admite: mirad la «Ayuda 3».

Qué se sube y cómo se evalúa. Se lee **una vez**, antes de entregar, y se vuelve a la rúbrica cuando el equipo revisa su trabajo. Mientras trabajáis no hace falta: para eso está el reto en una página.

- Volver al README del sprint · ficha del reto, para consultar.
- Vuestra hoja personal: autoevaluación y coevaluación.
- Cómo subir a GitHub paso a paso, con capturas: Cómo entregar en GitHub. Esta hoja responde a **dos preguntas distintas**, una tabla para cada una:

1 · Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-15/`**.

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`registro.csv`	**Lo creáis a partir de una plantilla.** Os damos `registro_experimentos_plantilla.csv`, solo con la cabecera	Copiarla como `registro.csv` y añadir **una fila por prueba**, de todas las personas. Es el mismo fichero que en el repositorio del proyecto se llama `experimentos/registro.csv`	**Sí** · `sprint-15/`
`03_modelos.ipynb`	**Lo creáis.** Parte del cuaderno de *baseline* del Sprint 14 y de las funciones del cuaderno `S15_01`	Escribir el código de los modelos, la comparación, la comprobación de fugas y el test; debe ejecutarse **de arriba abajo** con semilla fija	**Sí** · `sprint-15/`
`justificacion_candidato.md`	**Lo creáis.** No hay plantilla: el paso 4 os da la tabla	Una página: tabla comparativa, coste y beneficio, por qué gana el candidato, resultado en test y, al final, la sección «Revisión con la rúbrica»	**Sí** · `sprint-15/`
`modelo_candidato.joblib` (o `.keras` / `.pt` si el candidato es la red)	**Lo genera el cuaderno** al guardar el modelo ya elegido	Guardarlo una vez elegido el candidato; no se escribe a mano	**Sí** · `sprint-15/`
`metadatos.json`	**Lo genera el cuaderno** junto al modelo	Que recoja versión, fecha, métricas y variables; no se escribe a mano	**Sí** · `sprint-15/`

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`explicacion_modelo.png`	**Lo genera el cuaderno.** Gráfico de importancia de variables, SHAP o Grad-CAM	Guardar el gráfico y escribir 2–3 frases de lectura en la justificación	**Sí** · `sprint-15/`
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-15/`
Base de datos o carpeta local de MLflow (extra)	**Lo genera** MLflow si el equipo decide probarlo	Solo si el equipo llega al nivel extra; en la práctica es opcional	No se sube
Copias intermedias de cuadernos o del registro	**Las creáis** mientras probáis	Borrarlas o dejarlas fuera de `sprint-15/`	**No se suben**

2 · Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Partición común	Todos los modelos se comparan con **la misma partición**, definida y guardada antes de probar (temporal si hay series; por grupos si hay fotos del mismo objeto)	Mínimo	`03_modelos.ipynb`
Baseline en el registro	El *baseline* repetido es **la primera fila** del registro	Mínimo	`registro.csv`
Tres modelos clásicos	Al menos 3 modelos, con media ± desviación de la validación cruzada	Mínimo	`03_modelos.ipynb` y `registro.csv`
Modelo neuronal	Al menos un modelo neuronal con la misma partición (vale la red de scikit-learn `MLPClassifier`; la de Keras del Sprint 10 es opcional) o, en el brief 3, el asistente LLM/RAG	Estándar	`03_modelos.ipynb` y `registro.csv`
Registro de experimentos	Una fila por prueba con modelo, ajustes, métrica, desviación y tiempo; **al menos una fila de cada persona**; ≥ 12 filas recomendadas	Mínimo	`registro.csv`
Ajuste de hiperparámetros	1–2 ajustes clave de cada familia, probados **solo con validación** y anotados	Estándar	`registro.csv`
Control de fugas	Lista de 5 preguntas contestada en una celda de texto antes de elegir; nada del test ha servido para decidir	Estándar	`03_modelos.ipynb` (celda «Comprobación de fugas»)
Elección del candidato	Se elige con validación (nunca con el test) y se justifica en una página: tabla comparativa y coste y beneficio (tiempo, complejidad, explicabilidad)	Mínimo	`justificacion_candidato.md`
Test una sola vez	El candidato se evalúa en el test **una única vez**, después de elegir; si el resultado no gusta, se anota y se explica, **no se vuelve a elegir**	Mínimo	`03_modelos.ipynb` y `justificacion_candidato.md`
Modelo guardado	Modelo del candidato y `metadatos.json` (versión, fecha, métricas, variables)	Mínimo	`modelo_candidato.joblib` y `metadatos.json`

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Primera explicación	Importancia de variables, SHAP (tabular) o Grad-CAM (imágenes), con 2–3 frases de lectura. Se profundiza en el Sprint 16	Estándar	`explicacion_modelo.png` y `justificacion_candidato.md`
Reproducibilidad	Semilla fija, versiones de las bibliotecas anotadas y cuaderno ejecutable de arriba abajo; Git con al menos 3 *commits* de personas distintas, sin claves ni datos personales	Estándar	`03_modelos.ipynb` y historial del repositorio
Uso de la IA	Declarar qué se pidió y qué se revisó, y **marcar en el registro las pruebas hechas con código generado por IA** (por ejemplo, en `notas`)	Mínimo	`justificacion_candidato.md` y `registro.csv`
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa cuaderno, registro y justificación con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en la sección «Revisión con la rúbrica» de `justificacion_candidato.md`; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`justificacion_candidato.md` y `sprint-15/`
Extras	Hipótesis sobre **por qué gana** ese modelo, contrastada con la explicación · análisis de errores o curvas del candidato · MLflow en local	Extra	`justificacion_candidato.md` y `03_modelos.ipynb`

3 · Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de entregar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona y se ha medido bien?)	Hay modelos, pero las particiones no coinciden, se ha mirado el test para elegir o el cuaderno no se ejecuta	*Baseline* y 3 modelos con la **misma** partición y media ± desviación; candidato elegido con validación; test evaluado **una sola vez**	Además, un modelo neuronal (o LLM/RAG), hiperparámetros ajustados solo con validación, lista de fugas contestada, modelo guardado con `metadatos.json` y una explicación del modelo	Además, análisis de errores o curvas del candidato que enseñan **dónde falla**
Reproducibilidad (¿lo puede repetir otra persona?)	No hay semilla ni versiones, o el registro está vacío o sin ajustes	Semilla fija y versiones anotadas; el registro tiene las columnas rellenas	Otra persona ejecuta `03_modelos.ipynb` de arriba abajo y **obtiene las mismas cifras**; el registro tiene ≥ 12 filas con al menos una de cada persona; hay 3 o más *commits* de personas distintas, sin claves ni datos personales	Además, todas las filas del registro llevan la huella de los datos (`datos_hash`) y la semilla, y se anota el coste de las pruebas pesadas
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	La justificación solo dice «es el de mejor número»	Tabla comparativa con media ± desviación y candidato nombrado	Justificación de una página con coste y beneficio, dice si la diferencia con el siguiente modelo **puede ser ruido** y explica el resultado en test, sin jerga	Además, una hipótesis de **por qué gana** ese modelo, contrastada con la explicación, y una recomendación clara con sus límites

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
IA responsable (¿se sabe qué hizo la IA y qué comprobasteis?)	No dice si se ha usado IA	Dice que se ha usado IA y marca en el registro las pruebas con código generado	Dice **qué se pidió, qué se revisó y qué se cambió**; quien escribe sabe explicar cualquier línea; no hay claves ni datos personales en el repositorio	Además, cuenta un fallo concreto de la IA (por ejemplo, una fuga que no vio o que vio sin que existiera) y un límite de lo que dice la explicación del modelo

Cómo se revisa (unos 10 minutos, antes de subir; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar** `03_modelos.ipynb` de arriba abajo en un entorno limpio (por ejemplo, un Colab nuevo) y comparar las cifras con la tabla de la justificación. Anotar si algo no funciona o sale distinto.
2. **Mirar el registro**: ¿están el *baseline* en la primera fila, todas las columnas rellenas, una misma partición y al menos una fila de cada persona?
3. **Buscar fugas y trampas**: ¿se contestó la lista de fugas? ¿Aparece el test antes de elegir el candidato? ¿Hay alguna métrica «demasiado buena»? Que quien lo escribió lo explique.
4. **Leer la justificación como si fuerais el cliente**: ¿se entiende por qué gana ese modelo y si la diferencia puede ser ruido? Poner un nivel (1 a 4) en cada criterio, **con una frase de prueba** (por ejemplo: «3 · el cuaderno reproduce las cifras, pero falta la fila de Lucía en el registro»). Sin prueba escrita, el nivel no vale.
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del final de `justificacion_candidato.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.